

Programming Languages and .NET

Don Syme

MSR Cambridge

Principles of Programming Group



Today

1. Our .NET Framework Research

- We've been working with CLR & C# for ~2 years

2. Generics for .NET

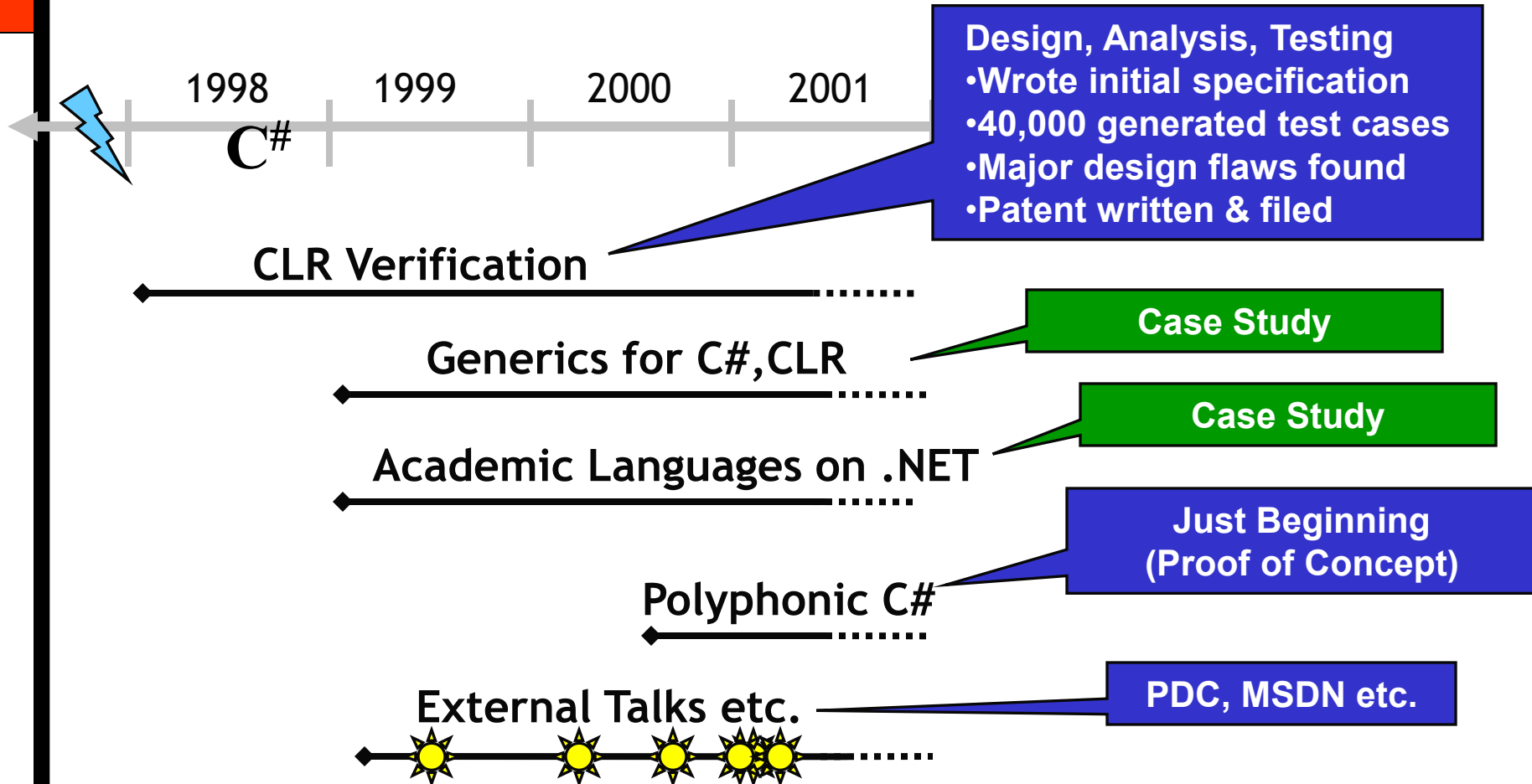
- Generics are designed, prototyped and ready

3. Academic Languages for .NET

- We're helping to successfully engage academic interest in the CLR
- With big benefits for Microsoft

Our Involvements with CLR & C#

50% of time for 8 Researchers, 1 SDE, 1 PostDoc



Today

Case Study: Generics for .NET

Generics: Status

1. A world first

- First generics for a JVM or CLR
- First cross-language generics
- First making full use of dynamic code generation

2. Design is suitable for C#, C++, VB, ML, Haskell...

3. Our CLR and C# prototypes are fast, robust and ready

Generics: Introduction

Generic Classes,
Generic Interfaces,
Generic Structs,
Generic Code

For:
Collection Classes,
Algorithms

Benefits:
Code Reuse,
Faster Code,
Safer Programming

```
public class Stack<T>
{
    public Stack() { nitems = 0; items = new T[10]; }
    public T Pop() { return items[--nitems]; }
    public void Push(T x) { ... ; items[nitems++] = x; }
    public bool IsEmpty() { ... }
    private T[] items;
    private int nitems;
}
```

C#

```
Stack<int> st;
..
switch (oper)
{
    case '+':
        st.Push (st.Pop() * st.Pop());
        break;
    case '*':
```

Using a Generic Class

Others agree...

C++

Ada

ML

Eiffel

Haskell

Mercury

**Component
Pascal**

Generics: A Good Thing

Ada, C++, Eiffel, ML, Haskell, Mercury, ...

Sun want it in Java, but can't get it

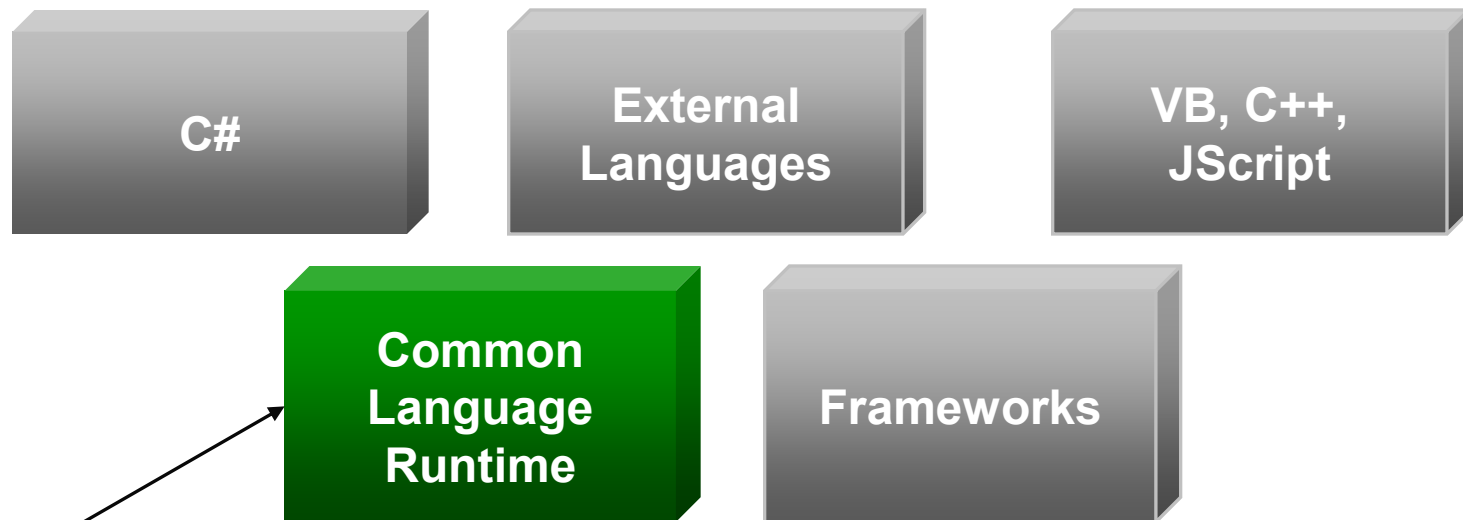
- Most popular feature requested
- 3 design attempts
- It requires modifications to the JVM to “do it right”

Anders to O'Reilly:

By having the Common Language Runtime understand the concept of generics, multiple languages can share the functionality.

MS devs all seem to agree

Generics: What We've Done

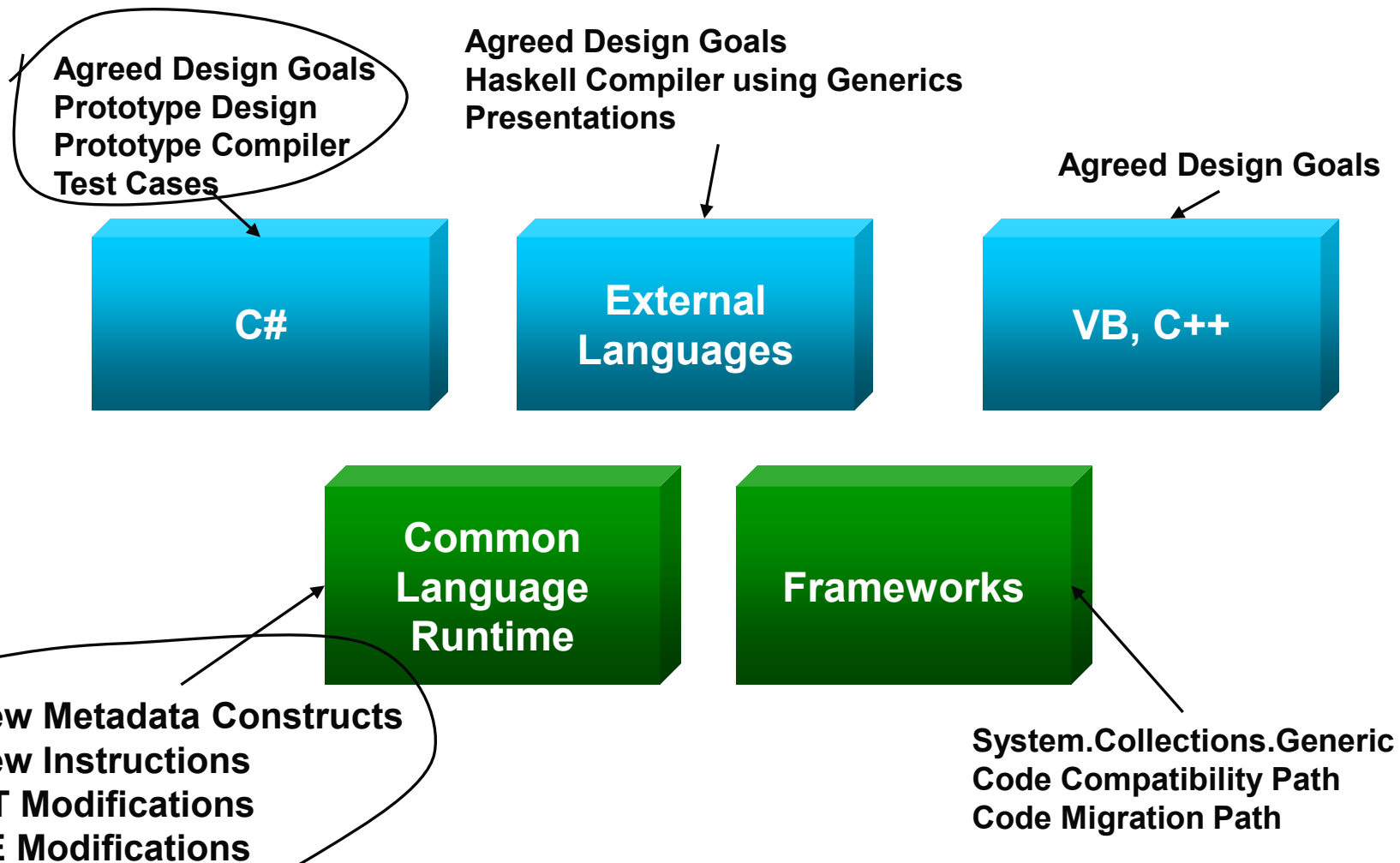


- Dynamically Load New Type Instantiations
e.g. `Stack<int>`
- Generates Code as Needed
- Shares Types, e.g. `Stack<int>` & `Stack<int>`
- Shares Code, e.g. code for `Stack<Object>` & `Stack<String>`

Generics: Design Comparison

	C++	Java with Generics (GJ)	C# with Generics
Dynamic Load	✗	✓	✓
Runtime Types	✓	✗	✓
List<int> Stack<Date>	✓	✗	✓
Efficient Execution	✓	✗	✓
Code Sharing	✗	✓	✓
Errors at compile time	✗ Link time	✓	✓
Ship without source	✗	✓	✓
Easy debug	✗ Name mangling	✗ No type info	✓

Generics: What We've Done



Implementation Summary

	<i>VS PDC (Aug 2000)</i>	<i>VS Beta-1 (Dec 2000)</i>	<i>VS Beta-2 (~Apr 2001)</i>	<i>If In Product</i>
Generic Classes (C#, CLR)	Done	Done	Done	Done
Generic Methods (C#, CLR)	Done	Done	Done	Done
Optimizations (CLR)	✗	Done	Done	Done
Shared Code for Classes (CLR)	✗	Done	Done	Done
Shared Code for Methods (CLR)	✗	✗	In Progress	Done
Constraints (C#, CLR)	✗	✗	In Progress	Done
Verifier, Debug (CLR)	✗	✗	In Progress	Done
Profile, E&C (CLR)	✗	✗	✗	For Teams
C# Ease-of-use VB, VC++ access, VS support, Collections.Generic	✗	✗	✗	For Teams

Generics: Proofs of Quality

Robustness

- Passes Beta 1 BVTs
- Now regularly running $>10^6$ lines of IL

Efficiency

- C# micro-benchmarks 6-10x faster
- One language (Haskell) is consistently 30% faster
- Faster where you'd expect, nothing runs slower

Generics: Summary

We've created a viable option....

- It's up to the product teams to exercise it...

Unfortunately not in V1

- Planning in progress for "Vnext"
- Possible CLR code-integration ~April 2001
- Some work for each team

Today

Academic Languages for .NET (Project 7)

Project 7: Aims

7 Academic, ~8 Commercial Languages

Developer Relations/University Relations

- James Plamondon, now Dan Fay, Brad Merrill

The Main Aims

- Multi-language momentum
- Robust compilers at RTM
- VS Integration at RTM

7 academic languages at PDC

On track for RTM

Some languages at RTM

Further Challenges

- Getting them working together
- Getting them running fast
- Develop an open, co-operative research community based around .NET, the CLR and multi-language programming

Beginning to Appear

Project 7: Our Contributions

1. ML for .NET

ML is popular with academics

- Wins competitions over C++, Java

Our Implementation

- High Quality
- Full libraries
- Excellent interop

2. Haskell for .NET

Haskell is very declarative

- Excellent for specification

Our Implementation

- Prototype

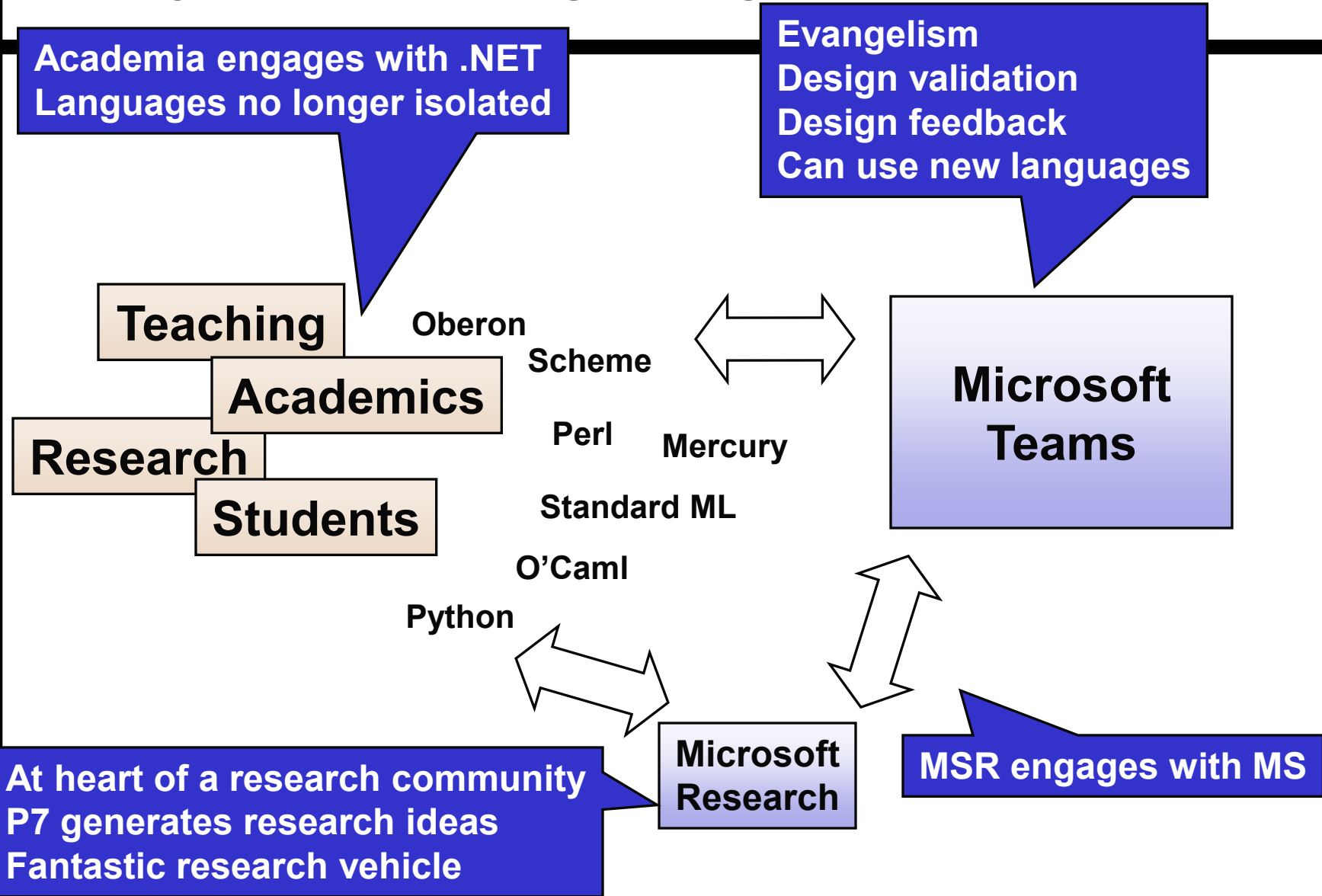
3. Technical Consolidation

4. Also

- Standardization efforts for inter-language working
- Prepare proposals for V2

- PDC, Workshops, Talks, Organization
- Product team/External Interaction, Publicity

Project 7: Everybody Wins



Summary

1. Our .NET Framework Research

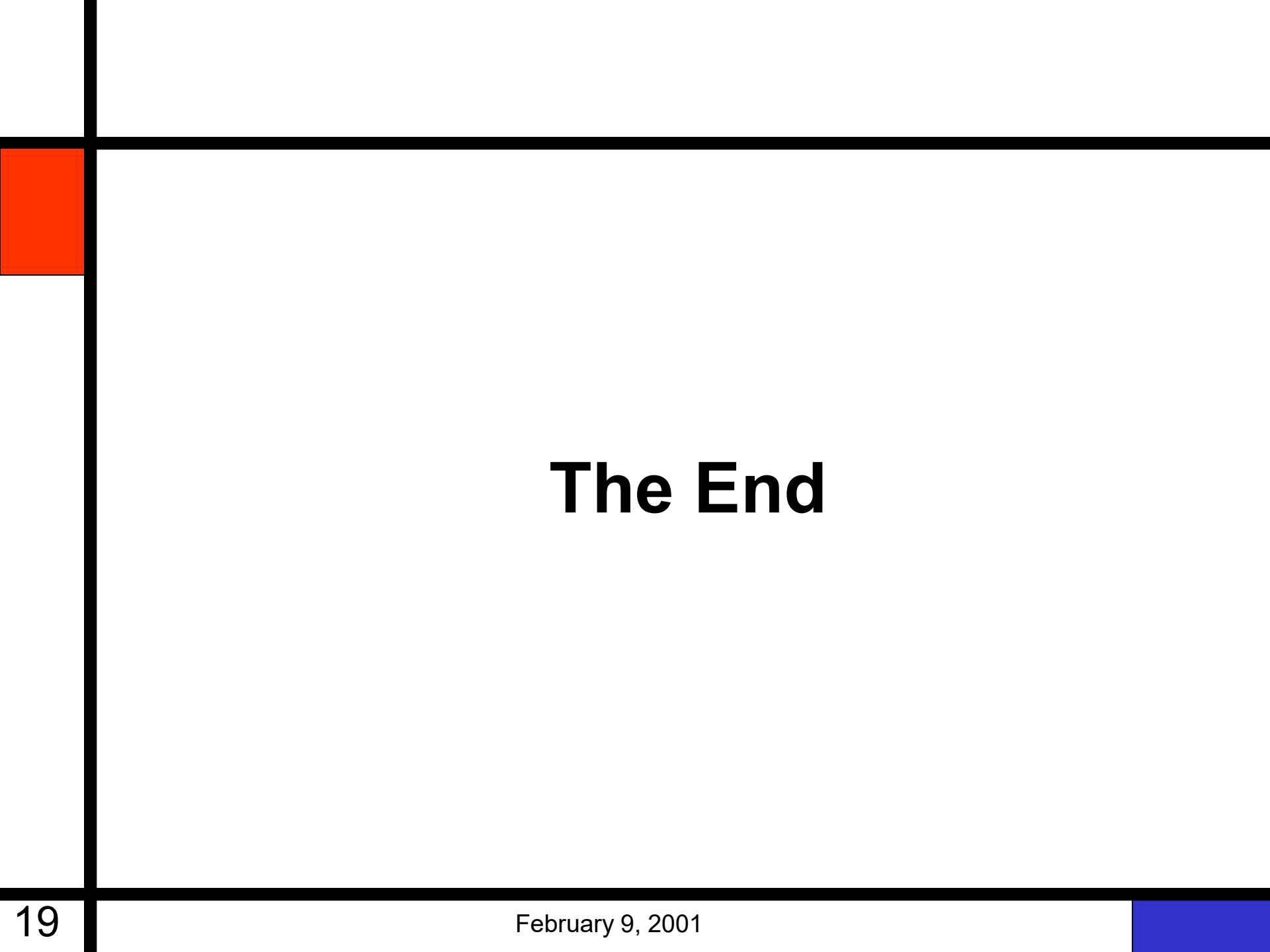
- We've been working with CLR & C# for ~2 years

2. Generics for .NET

- Generics are designed, prototyped and ready

3. Academic Languages for .NET

- We're helping to successfully engage academic interest in the CLR
- With big benefits for Microsoft



The End

Progress Beta1

<i>Language</i>	<i>Feed-back</i>	<i>Feasible?</i>	<i>Prototype</i>	<i>Demo</i>	<i>Perf Tests</i>	<i>Release</i>	<i>Consume CLS?</i>	<i>Produce CLS?</i>	<i>VS?</i>
Mercury	✓	✓	✓	✓	✓	✓	Planned		
Scheme(-)	✓	✓	✓	✓	✓	✓	Planned		
Eiffel(*)	✓	✓	✓	✓	✓	✓	✓	✓	✓
SML(+)	✓	✓	✓	✓	✓	Planned	✓		
Alice	✓	✓	✓	✓	✓	✓	Planned		
Oberon(*)	✓	✓	✓	✓	✓	✓	Planned		
Component Pascal	✓	✓	✓	✓	✓	Planned	Planned		
Mondrian	✓	✓	✓	✓		Planned	Planned		
Haskell	✓	✓	Underway				Planned		✓
Oz	✓	½	½						
Cilk	½								
O'Caml	✓	✓							

Standard ML

Simple Constructs

- Structured Datatypes (Records, Unions)
- Functions as Values
- Statically, Strongly Typed
- Garbage Collected
- Compiled

Explicit and Clear but not Verbose

- Type Inference the Key

Haskell

Similar Features to ML, Plus

- Delayed, “Pure” computations the norm
- Amazing optimizations possible

Programs as Mathematics, excellent for

- Component specifications
- Translation tools
- Testing

Explicit, Simple, Clear but not Verbose

- Type Inference the Key

Polyphonic C#

Language-level support for concurrency

- Clearer code and programmer intent
- Easier tool-based analysis for concurrency/orchestration bugs

Design for modern concurrency needs

- Handling multiple async messages/events
- Facilitating and optimizing thread use

Moving beyond Java

- Gain research mindshare

An “Obvious” MSR Future Project

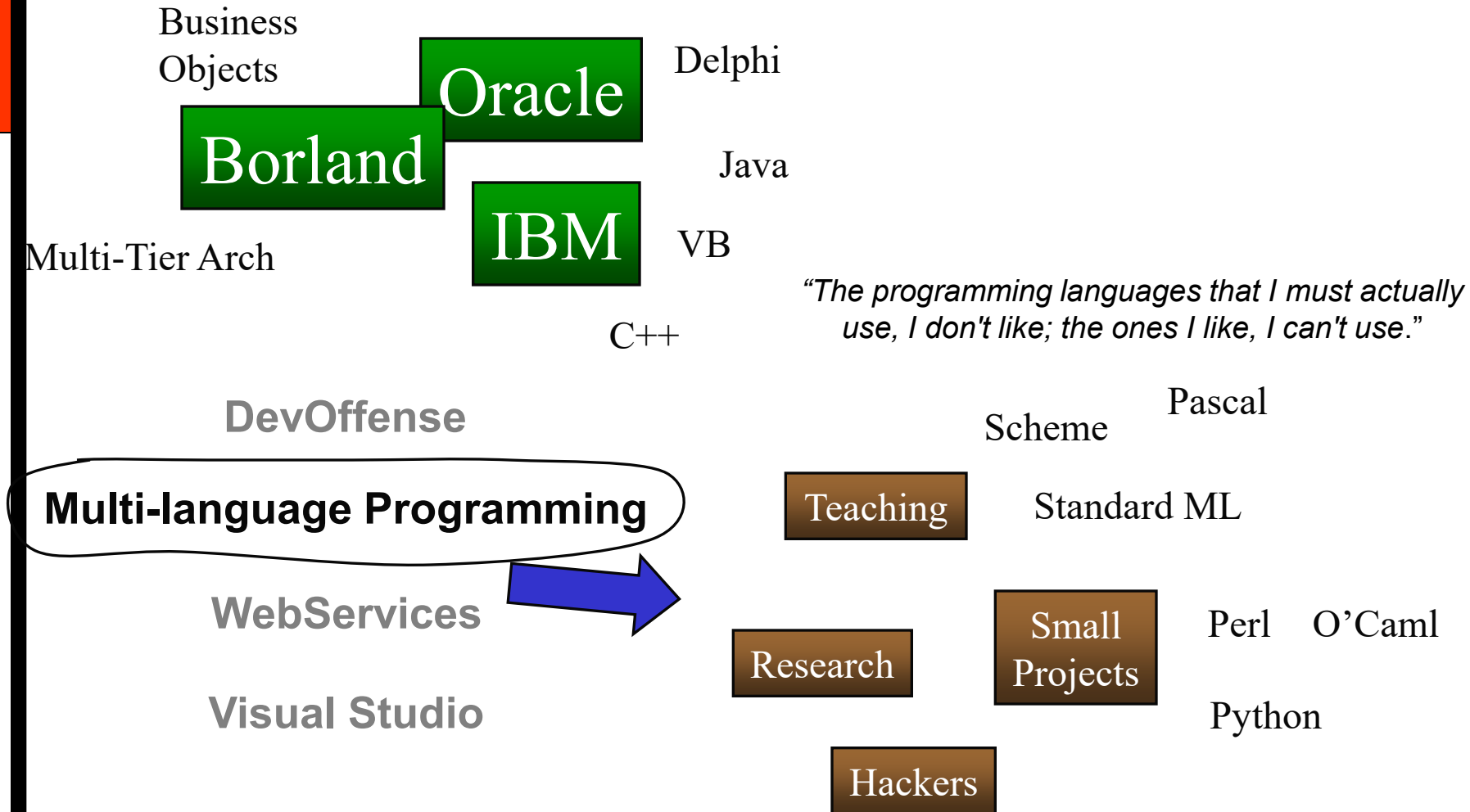
Advanced, more ambitious EE implementation project;

- Clearly should be MSR Redmond/CLR co-project;
- 2 year time scale, ~10 people?
- Advanced development in nature
- Must deliver faster, compatible implementation

Generics: Design Comparison

	<i>Java/C# via "Object"</i>	<i>C# with Generics</i>
Dynamic Load?	✓	✓
List<int>? Stack<Date>?	✗	✓
Efficiency (Alloc/Casts)	✗ Must allocate & cast at runtime	✓
Efficiency (Code Sharing)	✓	✓
Can find errors at compile time?	✗ Runtime	✓
Ship without source?	✓	✓

P7: Our Place in the Dev. Strategy?



Project 7: The Opportunity

“The programming languages that I must actually use, I don't like; the ones I like, I can't use.”

This attitude is very common: academics, students, hackers, “small-project” users, MSR researchers, MS devs....

- But .NET *can* run these languages
- It must be seen to do this at RTM

The aims:

- Academic, student, researcher, small project uptake of .NET teaching languages

P7: Good News, Good Opportunity

Good News

- 7 academic languages targeting .NET
- Project 7 is in place and is a GREAT idea
- VSIP is in place and is a GREAT idea
- CLR technically good enough, esp. with V2 features

The Opportunity

- At the moment, they won't be taken up for teaching
- Better implementations would lead to greater uptake
- Quality VSIP integration would lead to greater uptake
- Ask the academics: what do you *really* need